PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Практикум: текст как данные, частоты слов, признаки текста, ручная разметка и готовый шаблон классификации.

Работа строится как небольшой эксперимент: сначала формулируется правило разметки, затем создаётся базовая модель, после чего её решения проверяются на новых сообщениях. Такой порядок позволяет отделить корректность выполнения программы от качества данных и самой модели.

Паспорт занятия

Содержание

- Планируемые результаты
- Что понадобится
- 2. Вступление-обращение
- 3. Краткий теоретический минимум
- 4. Практический блок
- 5. Дополнительные задачи
- 6. Критерии оценивания
- 7. Чек-лист результата
- 8. Итог урока
- 9. Самопроверка
- Краткий глоссарий

Параметр	Описание
Название модуля	Алгоритмы и модели искусственного интеллекта
Номер и тема урока	3.14. Обработка текста как задача искусственного интеллекта
Формат	Практическое занятие под руководством преподавателя

Параметр	Описание
Продолжительность	2 академических часа (90 минут, включая короткий перерыв)
Целевая аудитория	10–11 классы, обучающиеся СПО
Основной результат	Умение представить текст числами, обучить простую модель и объяснить её ошибки

> **Сюжет практикума**
>
> Вы работаете в команде цифрового сервиса. В поддержку поступают короткие сообщения пользователей. Нужно автоматически распределять их по трём направлениям: «доступ», «оплата» и «доставка». Сначала вы разметите примеры вручную, затем запустите готовый шаблон классификатора и проверите, где он ошибается.
> Каждое сообщение должно получить ровно одну основную метку. Если в тексте затронуты сразу две темы, необходимо выбрать класс по заранее принятому правилу и отметить неоднозначность. Без такого правила разные разметчики могут назначить одному сообщению разные классы.

Планируемые результаты

- объяснять, почему компьютер не «понимает» текст напрямую и зачем нужны числовые признаки;
- выделять класс объекта и признаки текста;
- считать частоты слов и интерпретировать модель «мешок слов»;
- запускать готовый конвейер `CountVectorizer` → `MultinomialNB`;
- оценивать качество по `accuracy` и матрице ошибок;
- находить причины неверной классификации и улучшать обучающий набор.

Результат считается достигнутым, если полученные числа не просто записаны, а связаны с конкретными решениями модели: какие сообщения распознаны верно, какие классы перепутаны и какие особенности обучающих данных могли на это повлиять.

Что понадобится

- компьютер с браузером и средой Google Colab или Jupyter Notebook;
- Python 3 и библиотека scikit-learn в учебной среде;
- файл не требуется: набор данных встроен в шаблон занятия;
- рабочий лист или этот документ для записи выводов.

> **Организация работы**
>
> Код выполняйте индивидуально. Обсуждать причины ошибок модели можно в паре. Сохраняйте результаты запуска: значения метрик, таблицу предсказаний и краткие выводы. Не вводите в учебный набор реальные персональные данные.
> Ячейки тетради выполняйте сверху вниз. Перезапуск отдельной ячейки после изменения данных может оставить в памяти значения из предыдущего запуска, поэтому при сомнениях перезапустите среду и выполните все ячейки заново по порядку.

2. Вступление-обращение

> **Привет!**
>
> Сегодня вы увидите, как короткие фразы превращаются в числа и становятся входом для модели машинного обучения. Мы не будем писать классификатор с нуля: вы получите рабочий шаблон, запустите его, разберёте метрики и найдёте причины ошибок. К концу занятия вы сможете не только получить прогноз, но и аргументированно ответить, можно ли доверять модели.
> Особое внимание уделите пограничным сообщениям. Фраза может одновременно содержать слова об оплате и доступе, но модель всё равно должна выбрать один класс. Анализ таких случаев показывает, почему для оценки недостаточно увидеть только итоговую долю правильных ответов.

3. Краткий теоретический минимум

3.1. Текст как данные

Для человека фраза несёт смысл. Для алгоритма исходный текст — последовательность символов. Чтобы обучить модель, текст преобразуют в числа. Один текст обычно называют документом, отдельные элементы — токенами, а правильную категорию — меткой класса.
В этом практикуме объектом является одно обращение пользователя. Его входные данные — текст сообщения, а целевой результат — одна из трёх меток. На этапе обучения модель получает и тексты, и правильные метки; при прогнозировании ей передают только новые тексты.
Разделение на токены зависит от выбранного способа обработки. Например, слова могут быть приведены к нижнему регистру, а знаки препинания исключены. Поэтому токены не всегда полностью совпадают с тем, что человек визуально воспринимает как отдельные слова.

> **Аналогия**
>
> Представьте набор карточек. На каждой карточке написано сообщение пользователя и прикреплена цветная наклейка: синяя — «доступ», зелёная — «оплата», оранжевая — «доставка». Разметка — это прикрепление правильной наклейки.
> Во время обучения модель ищет повторяющиеся сочетания между содержимым карточек и цветами наклеек. Для новой карточки наклейки ещё нет: её цвет модель должна определить по найденным закономерностям.

> **Обратите внимание**
>
> Метка класса и признак — не одно и то же. Метка является правильным ответом, который модель должна предсказать, а признаки вычисляются из текста и используются для получения этого ответа. Например, частота токена «курьер» может быть признаком, а «доставка» — меткой класса.

3.2. Частоты слов и «мешок слов»

Один из простых способов представить текст — посчитать, сколько раз каждое слово встречается в документе. Порядок слов при этом частично теряется, поэтому подход называют «мешком слов» (bag of words). Результат — таблица чисел: строки соответствуют текстам, столбцы — словам, значения — частотам.

Словарь столбцов строится по обучающим текстам. Если словарь содержит токены «оплата», «курьер» и «пароль», то сообщение «курьер опаздывает» получит значение 1 в столбце «курьер» и 0 в столбцах «оплата» и «пароль». Остальные токены обрабатываются аналогично. Большинство значений такой матрицы обычно равны нулю: в одном коротком сообщении используется только небольшая часть общего словаря. Токен, которого не было при построении словаря, не создаёт новый столбец при обработке тестового текста и не даёт модели нового известного признака.

> **Ограничение подхода**
>
> Для простого мешка слов фразы «курьер не позвонил» и «курьер позвонил» различаются главным образом наличием токена «не». Связь между соседними словами не сохраняется как отдельный признак, если используются только униграммы. Поэтому одинаковые слова в разном порядке могут дать одинаковые наборы частот.

Рисунок 1. Конвейер простой классификации текста

Пример 1. Подсчёт частот

```
from collections import Counter
import re

texts = [
"оплата не проходит",
"оплата картой не проходит",
"оплата отклонена",
]

words = re.findall(r"[а-яё]+", " ".join(texts).lower())
print(Counter(words).most_common(3))
```

Регулярное выражение выбирает последовательности русских букв, а метод `lower()` приводит текст к нижнему регистру. Поэтому одинаковые слова, написанные с разным регистром, учитывались бы вместе. В данном примере ожидаемый результат — три наиболее частых токена: `[('оплата', 3), ('не', 2), ('проходит', 2)]`.

Проверьте понимание теории

1. Что будет выведено кодом из примера 1?

А) [(«оплата», 3), («не», 2), («проходит», 2)]
Б) [(«оплата», 1)]
В) Пустой список
Г) Ошибка

2. Почему слова «отправили» и «отправлен» могут стать разными признаками?

А) В простом векторизаторе это разные последовательности символов
Б) Они всегда имеют одинаковый индекс
В) Модель автоматически переводит их на английский
Г) Потому что один текст длиннее

3. Сравните два сообщения: «курьер не позвонил» и «курьер позвонил». Какой токен отличает их представления в простом мешке слов?

3.3. Готовый конвейер

В практикуме используется `Pipeline`: первый шаг `CountVectorizer` строит словарь и матрицу частот, второй шаг `MultinomialNB` обучается связывать эти частоты с классами. Модель не читает смысл как человек: она ищет статистические связи между словами и метками.
Конвейер нужен, чтобы одинаковая последовательность преобразований применялась и при обучении, и при прогнозировании. Метод `fit` сначала настраивает векторизатор на обучающих текстах, а затем обучает классификатор. Метод `predict` преобразует новые тексты с помощью уже построенного словаря и получает прогнозы классификатора.
Пример 2. Два шага в одном конвейере

```
model = Pipeline([
("vectorizer", CountVectorizer()),
("classifier", MultinomialNB()),
])
model.fit(train_texts, train_labels)
predictions = model.predict(test_texts)
```

> **Частая ошибка**
>
> Нельзя отдельно вызывать `fit_transform` на обучающем и тестовом наборах. В этом случае для тестовых данных может быть построен другой словарь и другие индексы

> столбцов. Правильная схема: обучить векторизатор на `train_texts`, а к `test_texts` применить только `transform`. В `Pipeline` эта последовательность выполняется автоматически.

3.4. Как оценивать качество

- Accuracy — доля верных предсказаний среди всех проверочных примеров.
- Матрица ошибок показывает, какие истинные классы модель перепутала и с какими классами.
- Precision отвечает на вопрос: насколько точны прогнозы выбранного класса.
- Recall отвечает на вопрос: какую долю объектов выбранного класса модель смогла найти.

Если из 12 проверочных сообщений 9 классифицированы верно, то `accuracy` равна 9 / 12 = 0,75. При этом одна и та же accuracy может соответствовать разным ошибкам: модель может понемногу ошибаться во всех классах или почти не распознавать один конкретный класс.
В матрице ошибок строки соответствуют истинным классам, а столбцы — предсказанным. Значения на главной диагонали показывают верные ответы. Числа вне диагонали указывают, сколько объектов одного класса было отнесено к другому.
Например, если модель предсказала класс «оплата» для четырёх сообщений, но только три из них действительно относятся к оплате, precision этого класса равна 3 / 4. Если в тестовом наборе было пять истинных сообщений класса «оплата», а модель нашла три, recall равна 3 / 5.

> **Важно**
>
> Одна высокая метрика не гарантирует, что система безопасна и полезна. Нужно смотреть на ошибки по классам и учитывать цену ошибки. Например, неверная маршрутизация обращения может увеличить время ответа пользователю.
> Метрики особенно осторожно интерпретируют на маленьком тестовом наборе. Здесь один исправленный прогноз заметно меняет итоговую долю, поэтому результат учебного эксперимента нельзя автоматически переносить на все будущие обращения.

4. Практический блок

> **Правило сложности**

> Задачи расположены по возрастанию сложности. Задача №1 — ситуация успеха, задача №5 требует анализа и улучшения модели. Для зачёта выполните все пять обязательных задач.
> Перед переходом к следующей задаче фиксируйте текущий результат. Это позволит сравнивать версии модели по одинаковому тестовому набору, а не опираться на впечатление от отдельных прогнозов.

Задача №1. Ручная разметка обращений

10 мин | 2 балл(а)

Прочитайте сообщения и определите класс. Запишите одно-два слова, которые повлияли на решение. Используйте классы: «доступ», «оплата», «доставка».
Выбирайте слова-подсказки из самого сообщения. Они должны объяснять решение, а не просто повторять название выбранного класса. Если одного слова недостаточно, можно указать короткое сочетание, например «код для входа».

№	Сообщение	Ваш класс	Слова-подсказки
1	Не приходит код для входа		
2	Курьер не успевает к назначенному времени		
3	С карты повторно списали сумму		
4	Хочу восстановить пароль		
5	Где посмотреть чек?		
6	Заказ ещё не передали курьеру		
7	После оплаты аккаунт оказался заблокирован		
8	Не могу изменить адрес доставки		
9	Платёж отклонён		

Контрольный вопрос: почему сообщение №7 может вызвать разногласия при разметке? Укажите две возможные темы сообщения и сформулируйте правило, по которому выбрана одна основная метка.

Задача №2. Слова превращаются в признаки

12 мин | 3 балл(а)

Запустите код. Затем ответьте на вопросы. Код показывает общие частоты слов в небольшом наборе обращений.
Общие частоты помогают увидеть состав набора, но не показывают, в каких именно документах встретилось слово. Поэтому частое слово не обязательно является хорошим признаком конкретного класса.

```
from collections import Counter
import re

sample_texts = [
"курьер опаздывает",
"курьер не позвонил",
"доставка задерживается",
"оплата не проходит",
"оплата картой отклонена",
]

def tokenize(text):
return re.findall(r"[а-яё]+", text.lower())

frequencies = Counter()
for text in sample_texts:
frequencies.update(tokenize(text))

print(frequencies.most_common(6))
```

> **Проверьте отступы перед запуском**
>
> В Python тело функции и тело цикла должны быть выделены отступами. Если при переносе листинга в тетрадь отступы перед строками после `def` или `for` не сохранились, среда сообщит об ошибке отступов. Восстановите стандартный отступ средствами редактора, не меняя сами команды.

1. Запишите шесть пар «слово — частота», которые вывела программа.
2. Назовите два слова, которые помогают распознать класс «доставка».
3. Почему слово «не» само по себе плохо различает классы?
4. Предложите одно улучшение токенизации или подготовки текста.

При ответе на третий вопрос сравните сообщения «курьер не позвонил» и «оплата не проходит»: один и тот же токен встречается в разных тематических классах.

Обязательное задание на анализ кода

1. Что произойдёт при выполнении фрагмента?

А) Будет выведен список классов
Б) Метод `transform` создаст словарь автоматически
В) Возникнет ошибка: векторизатор ещё не обучен
Г) Код всегда выведет нули

```
vectorizer = CountVectorizer()
X_train = vectorizer.transform(train_texts)
print(X_train.shape)
```

Объясните выбор, используя различие между методами `fit` и `transform`: первый находит словарь по данным, второй применяет уже найденный словарь.

Задача №3. Запуск готового шаблона классификации

20 мин | 4 балл(а)

Создайте новую тетрадь в Google Colab или Jupyter Notebook. Выполните ячейки по порядку. Не меняйте набор данных до получения базового результата.
Базовый результат нужен как контрольная точка. Если изменить данные до первого запуска, сравнить исходную и улучшенную версии по одному и тому же условию уже не получится.

Ячейка 1. Подготовленный набор данных

```
train_texts = [
"не могу войти в личный кабинет",
"забыл пароль от аккаунта",
"код подтверждения не приходит",
"учетная запись заблокирована",
"как изменить пароль",
"ошибка авторизации при входе",
"не удается зарегистрироваться",
"почта не принимается при входе",
"восстановите доступ к профилю",
"после обновления меня выбросило из аккаунта",
"логин не найден системой",
"двухфакторный код не работает",

"оплата картой не проходит",
"деньги списались два раза",
"как вернуть оплату",
"не могу оплатить подписку",
"платеж отклонен банком",
"где найти чек об оплате",
"с карты списали лишнюю сумму",
"возврат денег еще не пришел",
"нужно изменить способ оплаты",
"промокод не применился при покупке",
"оплатил заказ но статус не изменился",
"как получить счет для организации",

"где находится мой заказ",
"курьер опаздывает",
"посылка не пришла вовремя",
"как изменить адрес доставки",
"заказ передан в службу доставки",
"не могу выбрать пункт выдачи",
"когда привезут товар",
"трек номер не обновляется",
"доставка перенесена на другой день",
"курьер не позвонил",
"товар приехал в поврежденной коробке",
"как отменить доставку",
]
```

```
train_labels = (
["доступ"] * 12
+ ["оплата"] * 12
+ ["доставка"] * 12
)

test_texts = [
"не приходит смс с кодом для входа",
"аккаунт заблокировался после нескольких попыток",
"помогите восстановить пароль",
"после оплаты не могу войти в аккаунт",
"с меня повторно сняли деньги",
"почему платеж за подписку не проходит",
"нужен чек для бухгалтерии",
"оплатил покупку, но деньги вернулись",
"посылка задерживается уже третий день",
"хочу поменять адрес, куда приедет курьер",
"где посмотреть движение заказа",
"заказ оплачен, но его до сих пор не отправили",
]

test_labels = (
["доступ"] * 4
+ ["оплата"] * 4
+ ["доставка"] * 4
)
```

Обучающий набор содержит по 12 примеров каждого класса, всего 36 сообщений. Тестовый набор содержит по 4 примера каждого класса, всего 12 сообщений. Такое равное распределение упрощает учебный анализ, но в реальном потоке доли классов могут различаться.

Ячейка 2. Векторизация, обучение и оценка

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

class_names = ["доступ", "оплата", "доставка"]

model = Pipeline([
("vectorizer", CountVectorizer()),
("classifier", MultinomialNB()),
])

model.fit(train_texts, train_labels)
predictions = model.predict(test_texts)

print("Accuracy:", round(accuracy_score(test_labels, predictions), 2))
print("Матрица ошибок:")
print(confusion_matrix(test_labels, predictions, labels=class_names))
print("Отчет по классам:")
print(classification_report(
test_labels,
predictions,
labels=class_names,
zero_division=0,
))
```

Параметр `labels=class_names` задаёт одинаковый порядок классов в строках и столбцах матрицы. Поэтому первую строку и первый столбец следует читать как «доступ», вторые — как «оплата», третьи — как «доставка».

Зафиксируйте результат

Показатель	Ваш результат
Размер обучающего набора	
Размер тестового набора	
Accuracy	
Количество ошибок	

Показатель	Ваш результат
Класс с минимальным recall	

Количество ошибок можно определить двумя способами: посчитать несовпадения в таблице предсказаний или вычесть сумму диагональных элементов матрицы ошибок из общего количества тестовых сообщений.

> **Самопроверка запуска**
>
> Код должен выполниться без ошибок, вывести `accuracy`, матрицу ошибок 3 × 3 и отчёт по трём классам. Если результат не появился, сначала проверьте, что ячейка с набором данных была выполнена.
> Также проверьте, что количество текстов совпадает с количеством меток. Для обучения каждая строка из `train_texts` должна иметь соответствующую метку в `train_labels`.

Задача №4. Диагностика ошибок модели

15 мин | 5 балл(а)

Выведите все прогнозы и найдите неверные решения.
При анализе не ограничивайтесь поиском одного «виноватого» слова. На прогноз могут одновременно влиять знакомые модели токены, отсутствие важных токенов в словаре и частота классов в обучающем наборе.

Ячейка 3. Таблица предсказаний

```
for text, true_label, predicted_label in zip(
test_texts, test_labels, predictions
):
marker = "OK" if true_label == predicted_label else "ОШИБКА"
print(f"{marker:7} | {true_label:9} | {predicted_label:9} | {text}")
```

> **Проверьте отступы перед запуском**

> Строки, образующие тело цикла `for`, должны иметь отступ. Если форматирование листинга при переносе не сохранило отступы, восстановите их средствами редактора перед запуском.

5. Выпишите два текста, на которых базовая модель ошиблась.
6. Для каждого ошибочного текста укажите истинный и предсказанный класс.
7. Подчеркните слова, которые могли «перетянуть» прогноз в неверный класс.
8. Объясните, почему ошибка не обязательно означает ошибку в программном коде.

Программный код может корректно выполнить заданный алгоритм, но модель всё равно ошибётся из-за малого числа примеров, неоднозначной разметки или недостаточно информативных признаков. Различайте техническую ошибку выполнения и неверный прогноз обученной модели.

Текст	Истинный класс	Прогноз	Вероятная причина

Обязательное задание на исправление ошибок

В коде ниже есть одна техническая и одна логическая ошибка. Исправьте их и кратко объясните изменения.
Проследите жизненный цикл векторизатора: сначала он должен построить словарь по обучающим данным, затем использовать этот же словарь для преобразования тестовых данных. Матрицы `X_train` и `X_test` должны иметь одинаковое число столбцов и одинаковый смысл каждого столбца.

```
vectorizer = CountVectorizer()
X_train = vectorizer.transform(train_texts)
X_test = vectorizer.fit_transform(test_texts)

classifier = MultinomialNB()
classifier.fit(X_train, train_labels)
predictions = classifier.predict(X_test)
```

Подсказка: определите, на какой строке словарь ещё не существует, а на какой строке он ошибочно строится по тестовым данным.

Задача №5. Улучшение данных и сравнение качества

15 мин | 6 балл(а)

Добавьте в обучающий набор два примера, похожих на ошибочные тестовые тексты, но не копирующих их дословно. Затем переобучите модель и сравните качество.
Новые примеры должны сохранять смысл нужного класса и содержать естественные формулировки. Дословное копирование тестового текста сделало бы проверку некорректной: модель получила бы заранее тот пример, на котором затем оценивается.

Ячейка 4. Улучшенная версия

```
train_texts_improved = train_texts + [
"с меня снова сняли деньги",
"заказ еще не отправили",
]
train_labels_improved = train_labels + ["оплата", "доставка"]

improved_model = Pipeline([
("vectorizer", CountVectorizer()),
("classifier", MultinomialNB()),
])

improved_model.fit(train_texts_improved, train_labels_improved)
improved_predictions = improved_model.predict(test_texts)

print("Новая accuracy:", round(
accuracy_score(test_labels, improved_predictions), 2
))
```

После запуска полезно вывести не только новую accuracy, но и новые предсказания для тех же тестовых сообщений. Это покажет, какие именно решения изменились и не появилось ли новых ошибок вместо исправленных.

Сравнение	Базовая модель	Улучшенная модель
Количество обучающих текстов		
Accuracy		
Количество ошибок		
Изменение качества	—	

Сформулируйте итоговый вывод по шаблону:

> **Шаблон вывода**
>
> Базовая модель достигла accuracy _____ и допустила _____ ошибки. После добавления примеров accuracy стала _____. Качество изменилось, потому что _____. Однако результат нельзя считать окончательным, так как _____.

> **Важно**
>
> Отсутствие роста accuracy не означает, что эксперимент бесполезен. Новые примеры могли изменить отдельные прогнозы, но не итоговое число правильных ответов. Возможна и обратная ситуация: один старый прогноз исправился, а другой стал неверным. Поэтому сравнивайте и метрику, и список ошибок.

5. Дополнительные задачи

Эти задачи не влияют на обязательный результат занятия и предназначены для повышенной нагрузки.
Во всех дополнительных экспериментах сохраняйте тот же тестовый набор. Если одновременно изменить модель, обучающие данные и проверочные примеры, определить причину изменения качества будет сложно.

Дополнительная задача 1. Собственные обращения

Придумайте по два новых сообщения каждого класса. Получите прогнозы модели. Найдите хотя бы один пример, который можно понять по-разному, и объясните неоднозначность.
Не включайте созданные сообщения в обучение до первого прогноза. Иначе они перестанут быть новыми примерами для модели и не позволят проверить обобщение.

```
my_texts = [
# добавьте 6 собственных сообщений
]
print(improved_model.predict(my_texts))
```

Дополнительная задача 2. Униграммы и биграммы

Замените `CountVectorizer()` на `CountVectorizer(ngram_range=(1, 2))`. Сравните `accuracy`. Объясните, почему словосочетание «не приходит» может быть полезнее отдельных слов.

При параметре `ngram_range=(1, 2)` признаками становятся и отдельные токены, и пары соседних токенов. Это увеличивает словарь: «не» и «приходит» остаются отдельными признаками, а «не приходит» добавляется как ещё один признак. На малом наборе новые биграммы могут оказаться редкими, поэтому улучшение метрики не гарантировано.

Дополнительная задача 3. Цена ошибки

Выберите одну ошибку модели и опишите её возможные последствия для пользователя и службы поддержки. Предложите правило, когда модель должна передавать обращение человеку.

Правило должно опираться на наблюдаемое условие. Например, передача человеку может потребоваться для сообщения, в котором явно присутствуют признаки нескольких классов или для которого система не может уверенно выбрать одно направление.

6. Критерии оценивания

Задача	Проверяемый результат	Баллы
№1	Размечены 9 сообщений, указаны слова-подсказки, объяснена неоднозначность	0–2
№2	Запущен подсчёт частот, корректно интерпретированы признаки и ограничение слова «не»	0–3
№3	Шаблон запущен, зафиксированы метрики и размеры наборов	0–4
№4	Найдены ошибки модели, объяснены причины, исправлен код	0–5

Задача	Проверяемый результат	Баллы
№5	Модель улучшена, результаты сравнены, вывод содержит ограничение эксперимента	0–6
Итого	Максимум	20

> **Шкала**
>
> 18–20 баллов — высокий уровень; 15–17 — хороший; 12–14 — базовый результат достигнут; менее 12 — требуется доработка. Дополнительные задачи могут учитываться преподавателем как бонус, но не заменяют обязательные задачи.
> Полный балл предполагает не только работающий код, но и объяснение результатов. Записанная метрика без анализа ошибок не подтверждает понимание того, как модель принимает решения.

7. Чек-лист результата

☐ Я могу объяснить, что такое документ, токен, признак и класс.
☐ Я вручную разметил примеры и заметил неоднозначный текст.
☐ Я запустил подсчёт частот слов.
☐ Я выполнил готовый шаблон `CountVectorizer` → `MultinomialNB`.
☐ Я записал `accuracy` и прочитал матрицу ошибок.
☐ Я нашёл неверные прогнозы и объяснил вероятные причины.
☐ Я улучшил обучающий набор и сравнил результаты.
☐ Я не использовал реальные персональные данные.

Если какой-либо пункт не отмечен, вернитесь к соответствующей задаче и дополните результат конкретным доказательством: выводом программы, заполненной таблицей или кратким объяснением.

8. Итог урока

> **Сегодня вы сделали полный мини-цикл классификации текста**
>
> Разметили данные → превратили слова в частоты → обучили модель → проверили качество → разобрали ошибки → улучшили набор данных. Главная идея: модель

> классифицирует текст не «по смыслу вообще», а на основе числовых признаков и закономерностей, которые были представлены в обучающих примерах.
> Качество каждого этапа влияет на следующий: непоследовательная разметка создаёт противоречивые примеры, ограниченные признаки теряют часть информации, а маленький тестовый набор даёт нестабильную оценку. Поэтому работу с текстовой моделью нельзя свести только к вызову метода `fit`.

Кратко сформулируйте один вывод о данных и один вывод о модели. В первом укажите, каких обучающих примеров не хватало, во втором — какое ограничение мешка слов проявилось в эксперименте.

---

9. Самопроверка

1. Что делает `CountVectorizer`?

---

2. Для чего нужна ручная разметка?

---

3. Что показывает `accuracy`?

---

4. Почему тестовые тексты не следует использовать для обучения до оценки?

---

5. Как называется таблица, показывающая перепутанные классы?

---

6. Что лучше сделать после обнаружения систематической ошибки?

---

7. Верно ли утверждение: «Если accuracy стала 1,00 на 12 примерах, модель гарантированно идеальна»?

---

8. Почему для тестовых текстов используют словарь, построенный на обучающем наборе?

---

9. Где в матрице ошибок находятся правильные прогнозы?

Краткий глоссарий

Термин	Простое объяснение
Документ	Один отдельный текстовый объект: сообщение, отзыв, письмо.
Токен	Элемент текста, обычно слово или часть слова.
Признак	Числовая характеристика, которую получает модель.
Разметка	Назначение каждому примеру правильного класса.
Класс	Категория результата, например «доступ».
Словарь	Набор токенов, найденных в обучающих текстах.
Мешок слов	Представление текста через частоты слов без полного учёта порядка.
Векторизация	Преобразование текста в числовой вектор признаков.
Униграмма	Один токен, используемый как отдельный признак.
Биграмма	Пара соседних токенов, используемая как отдельный признак.
Обучающий набор	Тексты с известными метками, по которым модель настраивает свои параметры.
Тестовый набор	Отложенные примеры, не использованные при обучении и предназначенные для проверки качества.
Accuracy	Доля верных прогнозов среди всех проверенных примеров.
Precision	Доля правильных ответов выбранного класса среди всех прогнозов этого класса.
Recall	Доля найденных объектов выбранного класса среди всех истинных объектов этого класса.
Матрица ошибок	Таблица истинных и предсказанных классов.
Конвейер	Последовательность связанных шагов обработки данных и работы модели.
Переобучение	Ситуация, когда модель слишком хорошо запомнила учебные данные и хуже работает на новых.